



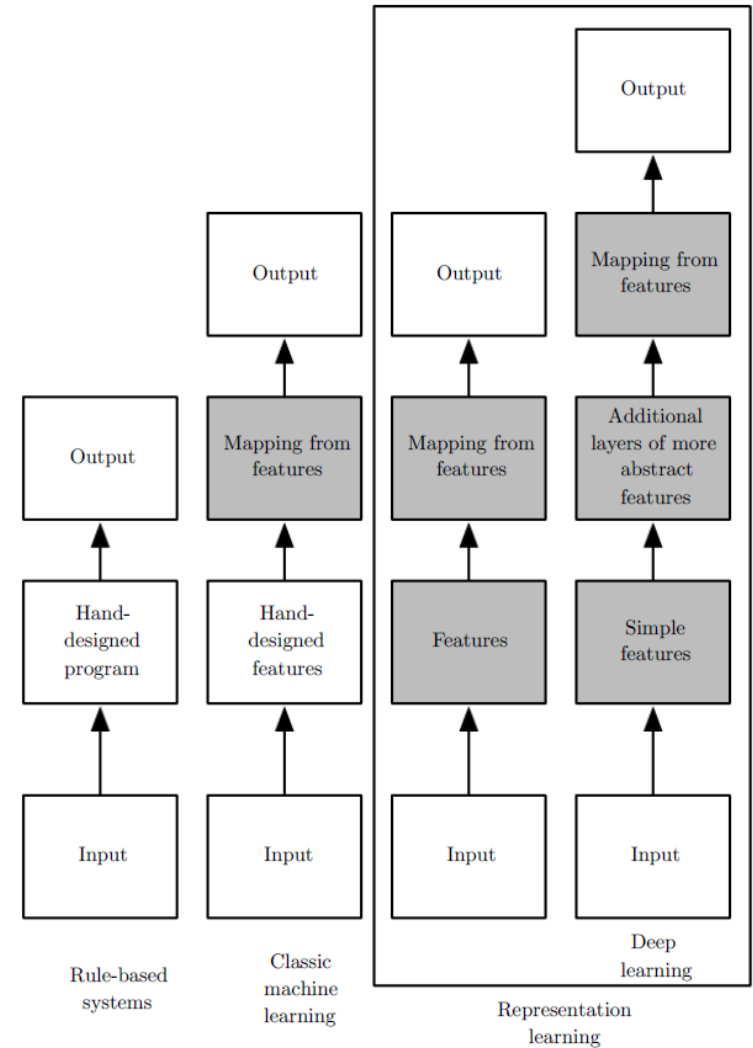
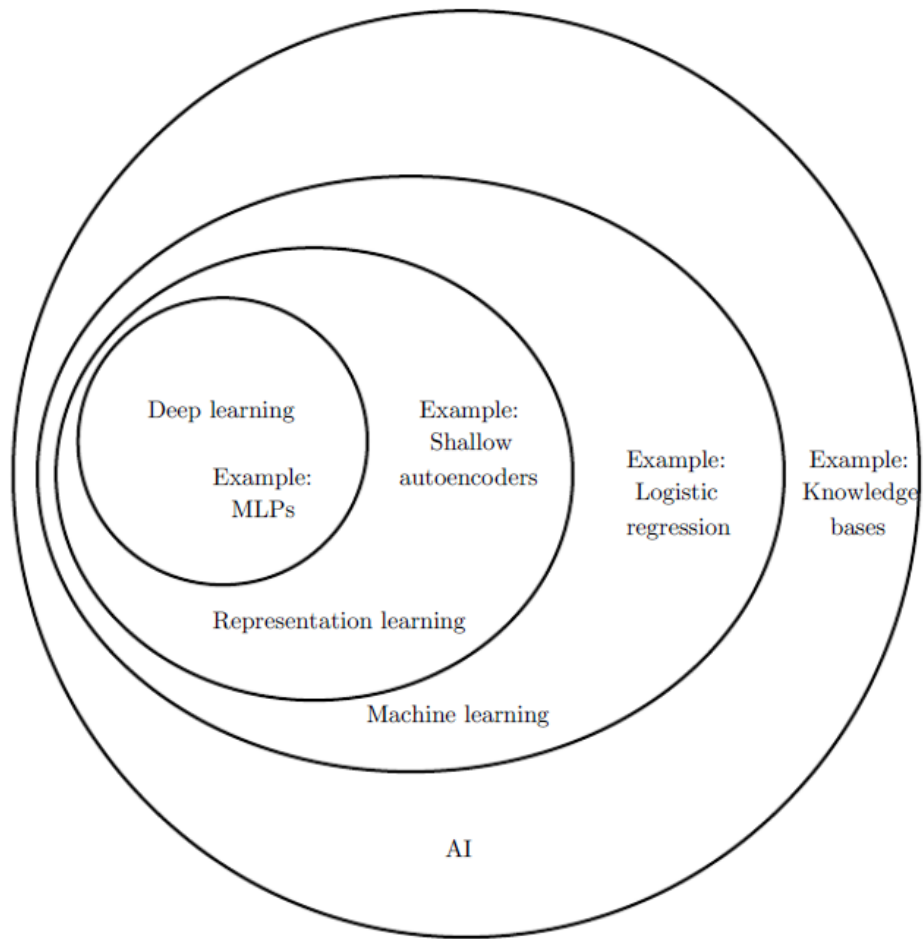
Deep Convolutional Network

Chao-Tsung Huang

**National Tsing Hua University
Department of Electrical Engineering**

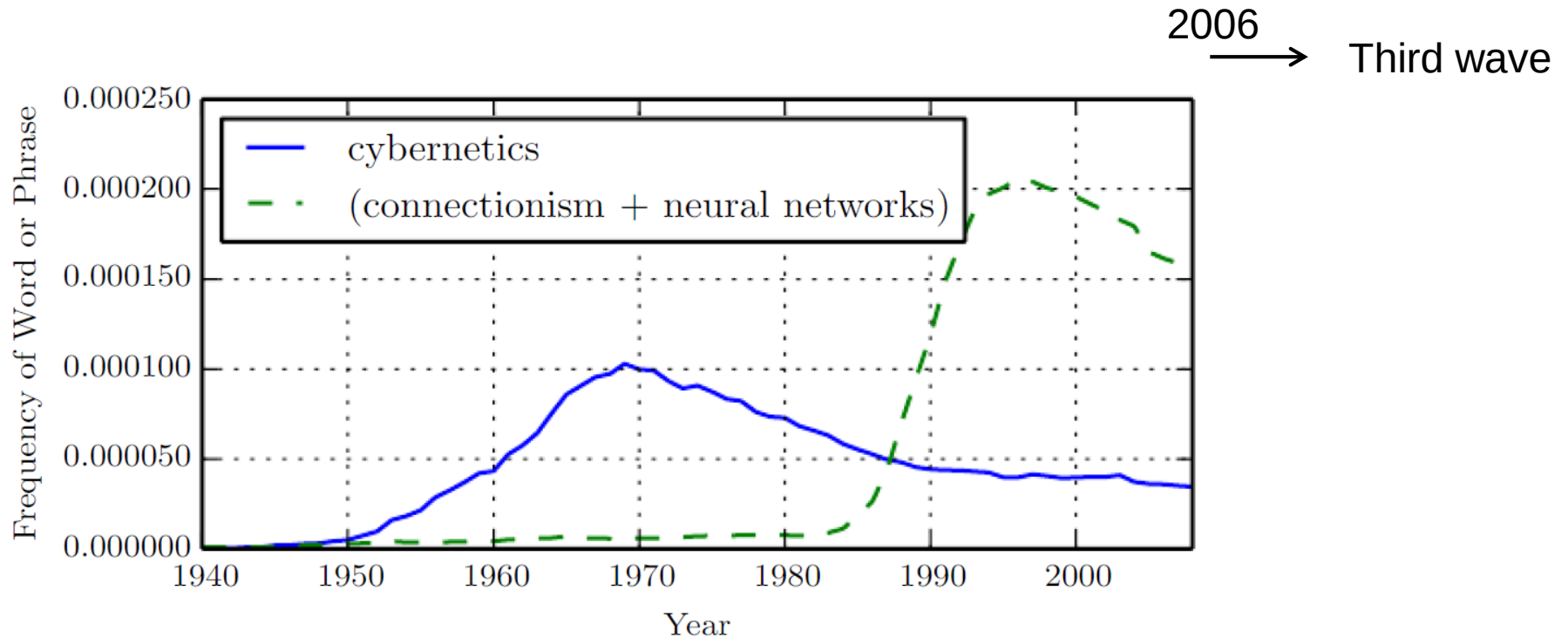
Ref: I. Goodfellow et al, *Deep learning*, The MIT Press, 2016.

Categories of learning



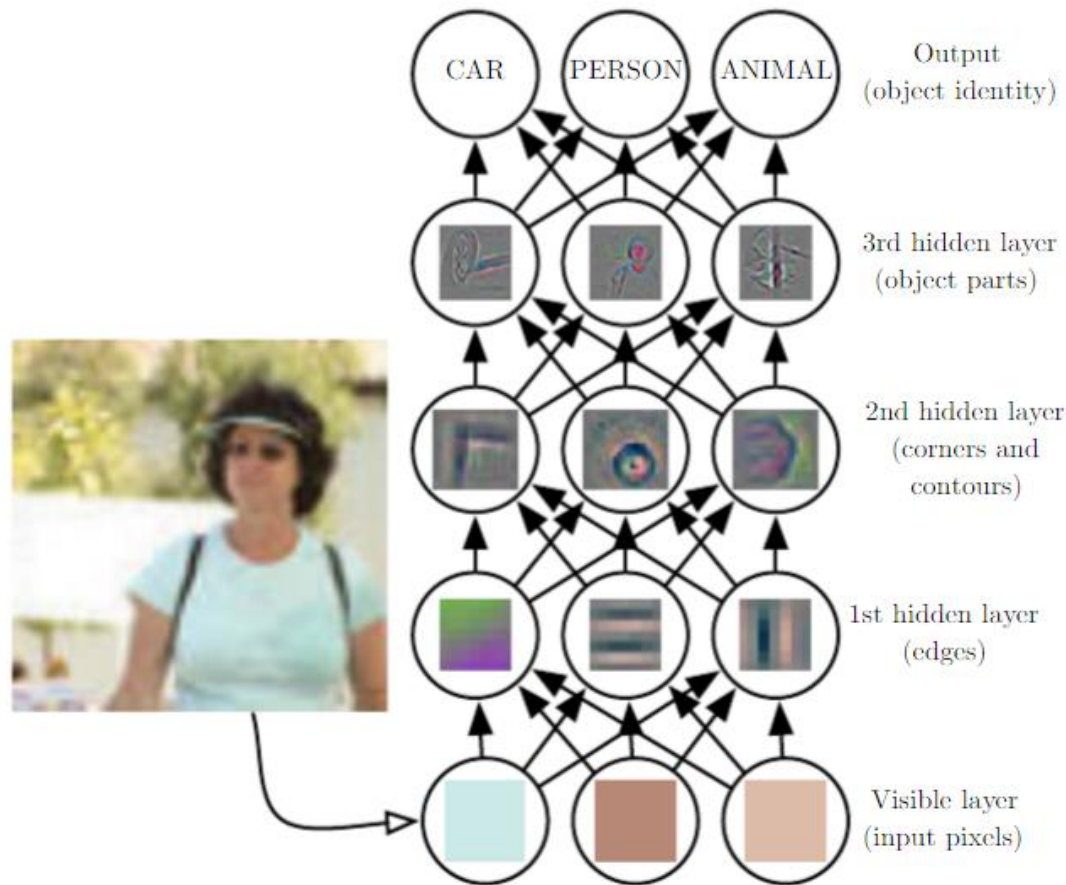


Three waves of deep learning



Convolutional neural network (CNN)

- Deep feedforward networks, in particular useful for visual images





Outline

- **Machine learning basics**
- Deep feed-forward network training
- CNN evolution for image recognition



Toy example: Linear regression

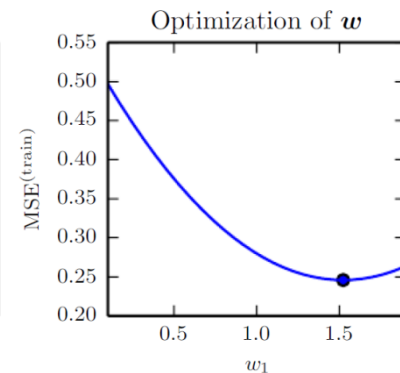
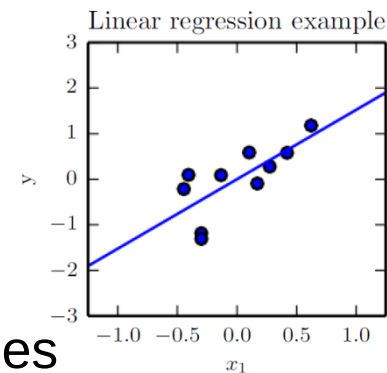
- Goal: minimize test error by learning hypothesis model over training data
- Task $\mathcal{T}$: Predict y from $\mathbf{x}$ using linear formulation

- Training

- Hypothesis $\mathcal{K}$: $\hat{y} = \mathbf{w}^T \mathbf{x} + b$
 - Model parameter: weight $\mathbf{w}$ and bias b
- Loss $\mathcal{L}$: mean squared error (MSE)
- Experience $\mathcal{E}$: training $\{\mathbf{x}, y\}$ examples
- Algorithm $\mathcal{A}$: supervised learning
 - Analytic solution in this example; gradient-based in deep learning

- Test

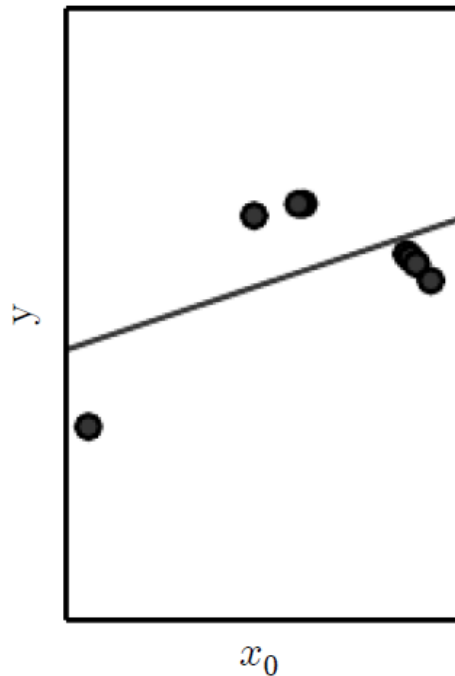
- Performance metric $\mathcal{P}$: Maximum Error
 - $\mathcal{P}$, usually not differentiable, not equal to $\mathcal{L}$
 - Generalization error (test error) on *unseen* test data



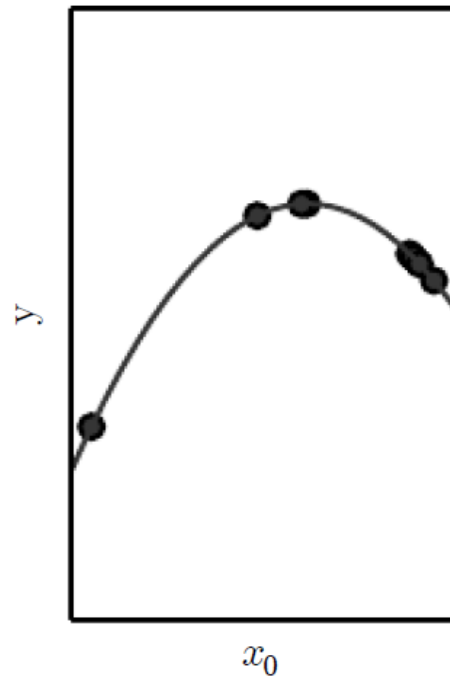
Capacity

- Model capacity: ability to fit a wide variety of functions
 - Related to hypothesis space (parameter space)

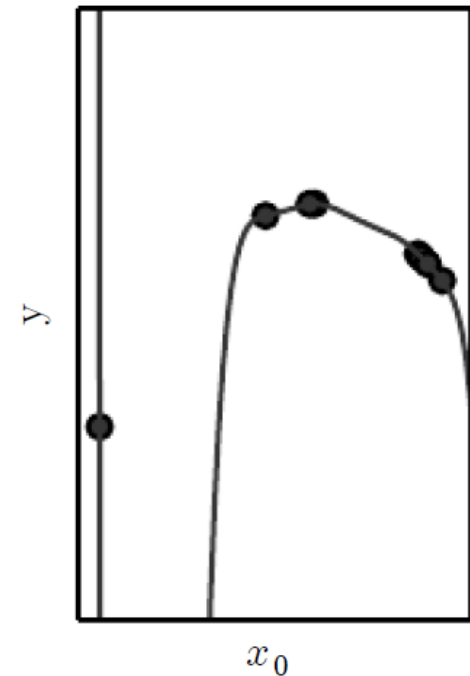
Underfitting



Appropriate capacity

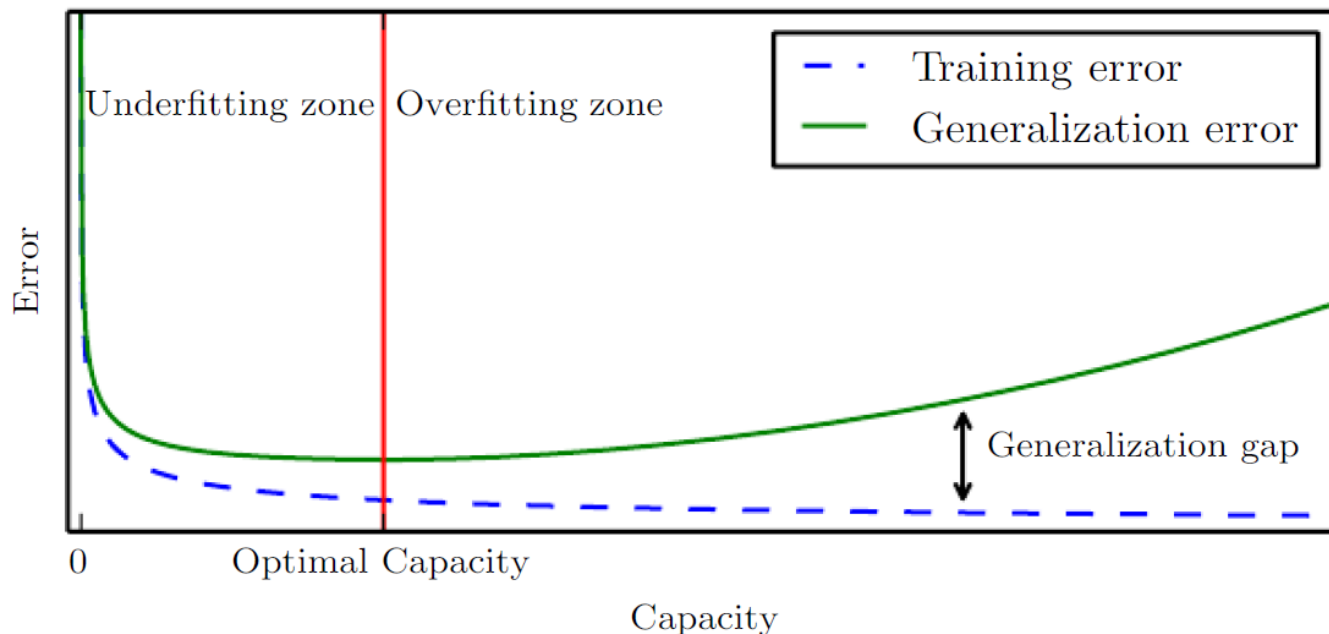


Overfitting



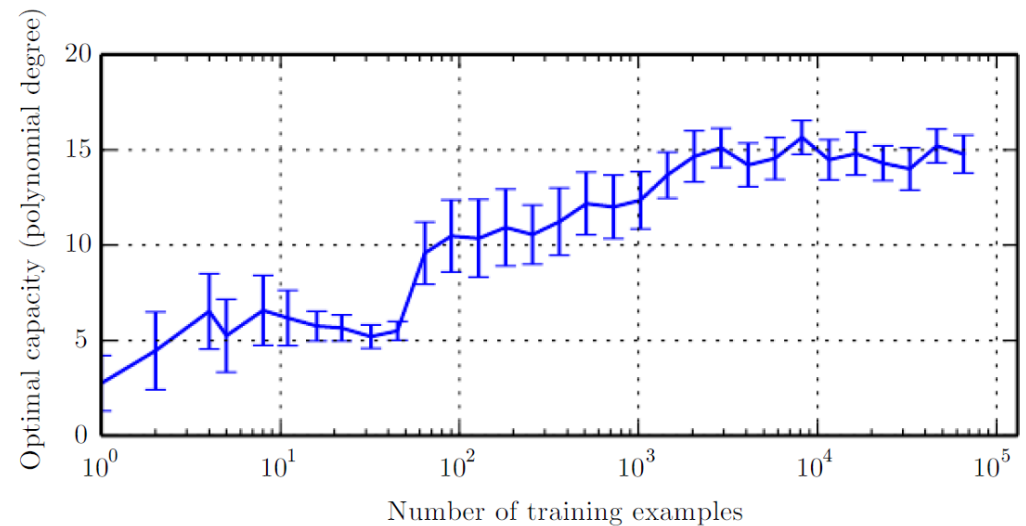
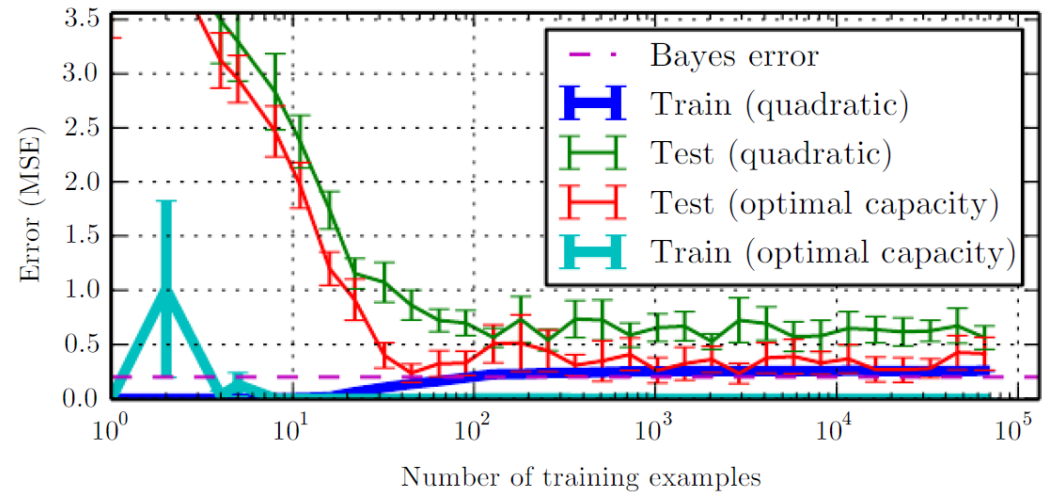
Underfitting and overfitting

- Underfitting: large training error
- Overfitting: small training error but large test error
- Appropriate fitting: small training error and small generalization gap



Optimal capacity

- Optimal capacity depends on the problem and training dataset size
 - Example: data from adding noise to a degree-5 polynomial



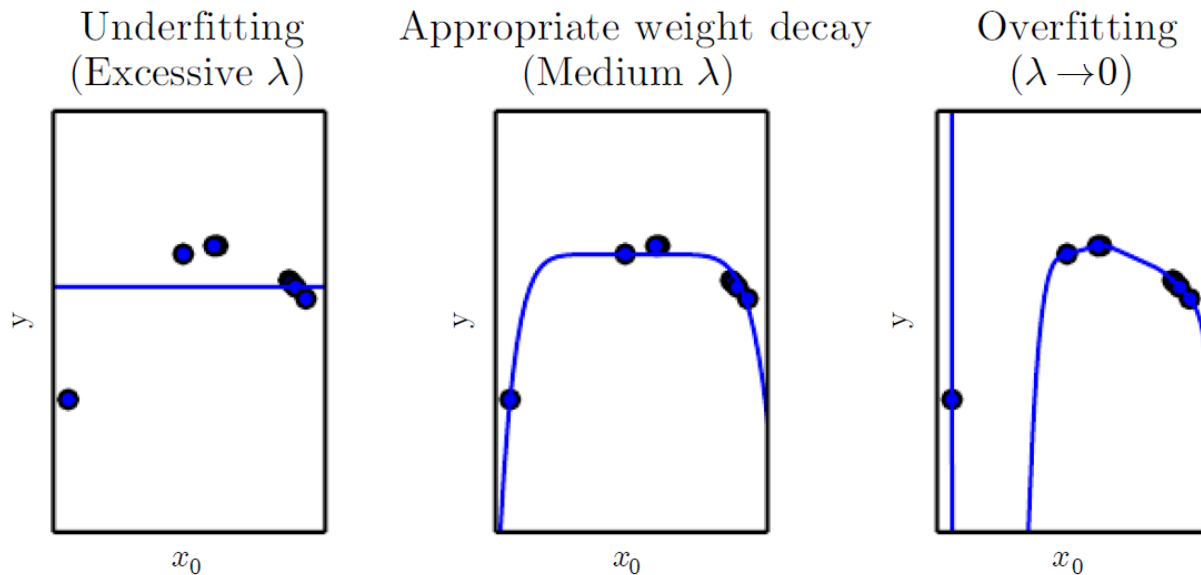


No free lunch theorem

- “Averaged over all possible data, every learning algorithm is similarly bad for new data”
 - Reason: Learning is based on abductive reasoning; inferring a general rule from limited examples is logically invalid
- Implication for real problems
 - Have to make sure that training and test data belong to similar distribution (for a specific task)
 - Remember that abduction cannot replace induction and deduction

Regularization

- Human preference or prior
 - for specific tasks (task-wise assumption)
 - to control model capacity
 - to reduce generalization error but not training error
 - Example: weight decay $J(\mathbf{w}) = \text{MSE}_{\text{train}} + \lambda \mathbf{w}^\top \mathbf{w}$





Hyper-parameter and validation set

- Hyper-parameter
 - Settings to control the algorithm's behavior and not adapted by the algorithm itself
 - Example: weight decay λ , training iterations, learning rate, ...
- Validation set
 - Dataset for selecting better hyper-parameters, e.g. $\lambda = 0, 1$, or 5 ?
 - Always constructed from training data
 - Not test data; not seen during parameter training
 - Note: competing networks may overfit to particular datasets, e.g. ImageNet, by using test sets as validation sets

Training data:	Learn model parameters by minimizing <i>loss</i>
Validation data:	Decide hyper-parameters by minimizing <i>error</i>
Test data:	Test generalization error (of infinite size in general)

Deep learning vision tasks

Classification



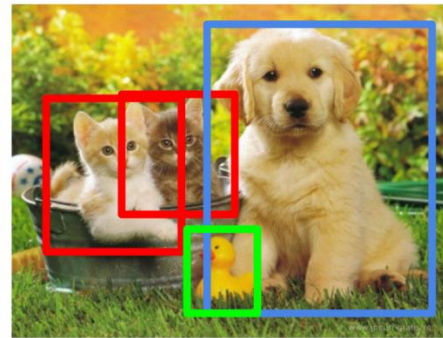
CAT

**Classification
+ Localization**



CAT

Object Detection



CAT, DOG, DUCK

**Instance
Segmentation**



CAT, DOG, DUCK

Single object

Multiple objects

credit: <https://analyticsindiamag.com>

Deep learning vision tasks

Monet $\leftrightarrow$ Photos



Zebras $\leftrightarrow$ Horses



Style Transfer

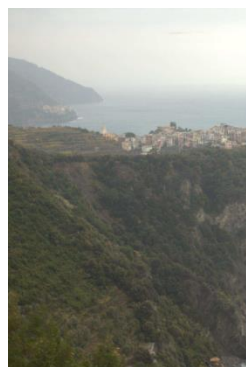
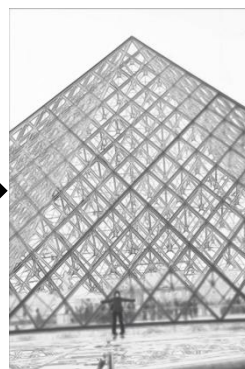


Image Processing

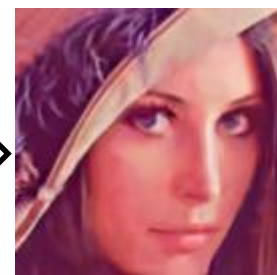
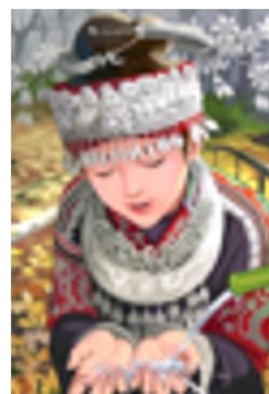


Image Restoration

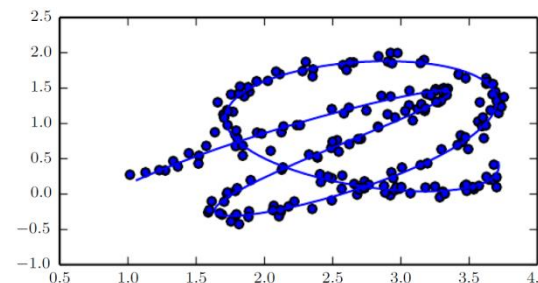


Challenges motivating deep learning (third wave)

- Curse of dimensionality
 - data dimension $|\mathcal{D}| \nearrow \Rightarrow$ hypothesis space $|\mathcal{K}| \nearrow \Rightarrow$ training set size $\nearrow$
 - Machine learning becomes exceedingly difficult
- Solution provided by deep learning
 - Efficient algorithm and powerful hardware to learn from large training set, i.e. 1.2M images in ImageNet
 - Hierarchical feature decomposition $\Rightarrow$ reduce $|\mathcal{K}|$
 - Manifold learning $\Rightarrow$ reduce $|\mathcal{D}|$

What DL can't do so far:

1. Tasks without input-output association
2. Tasks too difficult for human to verify

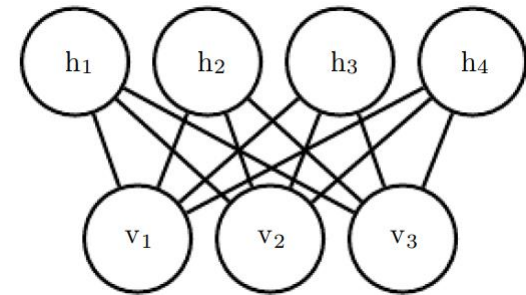
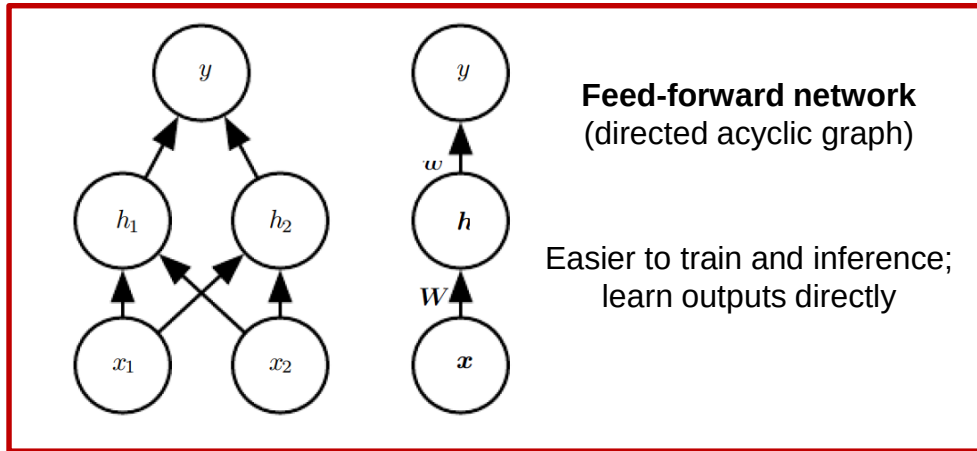




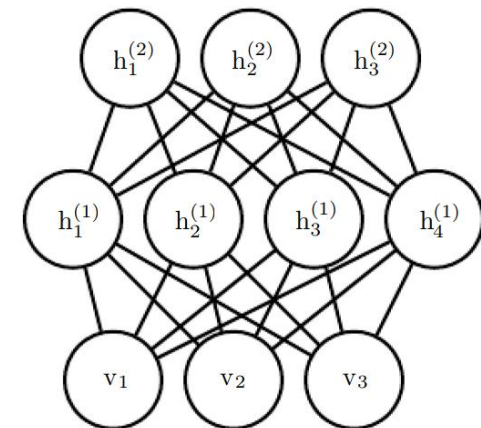
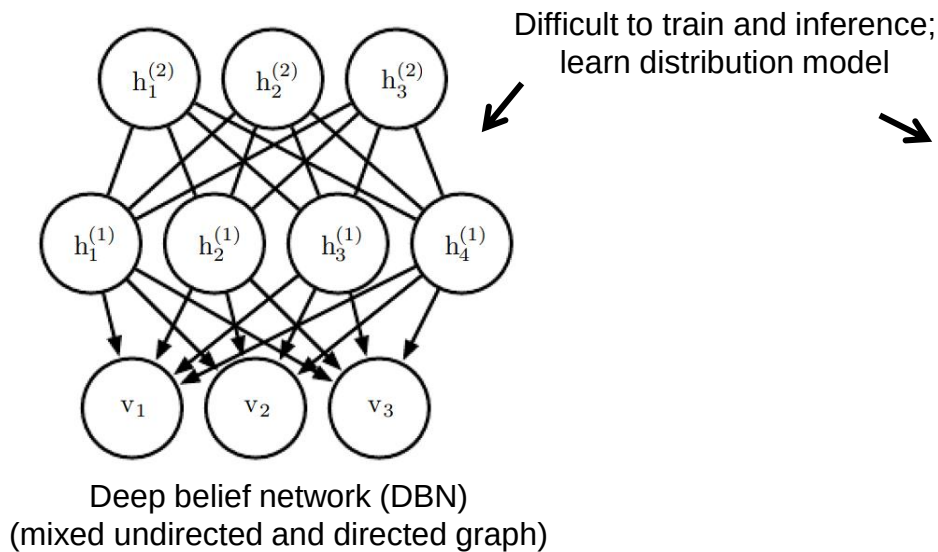
Outline

- Machine learning basics
- **Deep feed-forward network training**
 - Network architecture
 - Gradient-based learning
 - Back-propagation
 - Regularization and optimization
- CNN evolution for image recognition

Feed-forward vs. Boltzmann



Restricted Boltzmann machine (RBM)
(undirected bipartite graph)



Deep Boltzmann machine (DBM)
(multi-layer RBM)



Feed-forward network architecture

- Multi-layer perceptron (MLP)
 - One depth layer means one hidden layer or one output layer
 - Universal approximation theorem
- Hidden layer
 - Linear computation followed by non-linear activation
 - Previously popular: ReLU
 - Currently popular: GeLU, SiLU
 - Others: leaky ReLU, sigmoid, tanh, ...

$$h = g(\mathbf{W}^\top x + b)$$



Feed-forward network architecture

- Output layer

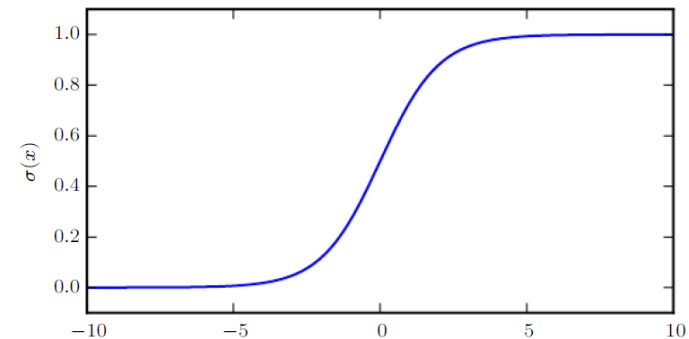
- Linear unit: regression output, distribution mean output

$$\hat{y} = \mathbf{W}^\top \mathbf{h} + b$$

- Sigmoid unit: probability output for binary variable y , $P(y=1)$

$$\hat{y} = \sigma(\mathbf{w}^\top \mathbf{h} + b)$$

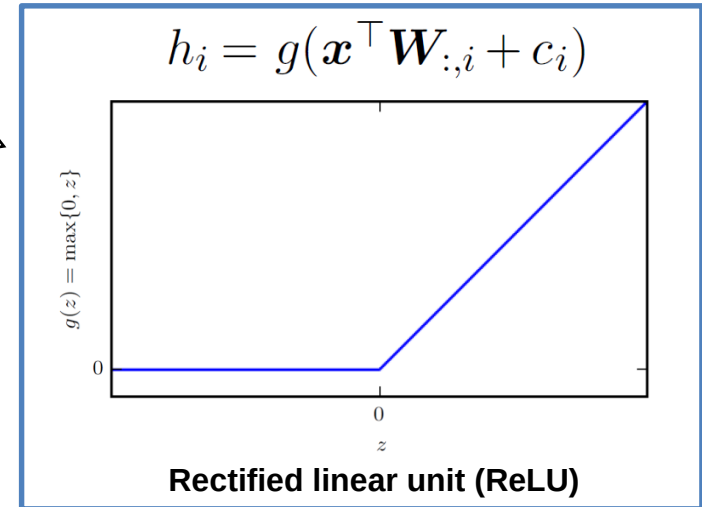
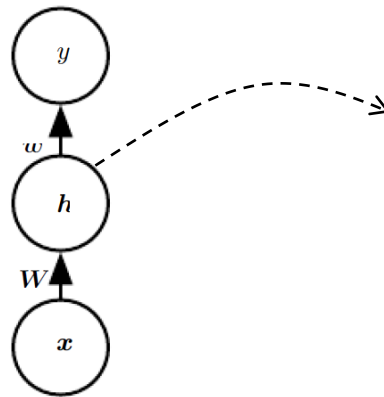
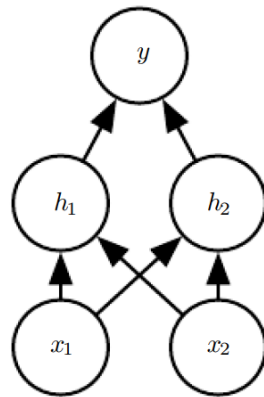
$$\sigma(x) = \frac{1}{1 + \exp(-x)}$$



- Softmax unit: probability output for multi-noulli variable, $P(y=i)$

$$\text{softmax}(\mathbf{z})_i = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \quad \mathbf{z} = \mathbf{W}^\top \mathbf{h} + b$$

Example: Learning XOR

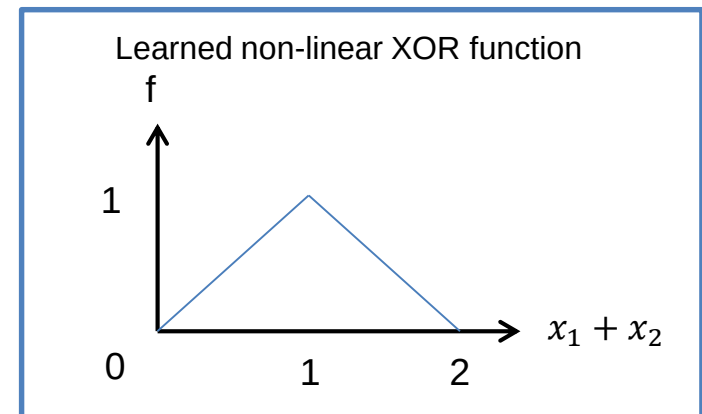


$$f(\mathbf{x}; \mathbf{W}, \mathbf{c}, \mathbf{w}, b) = \mathbf{w}^\top \max\{0, \mathbf{W}^\top \mathbf{x} + \mathbf{c}\} + b$$

Ideal learned parameters

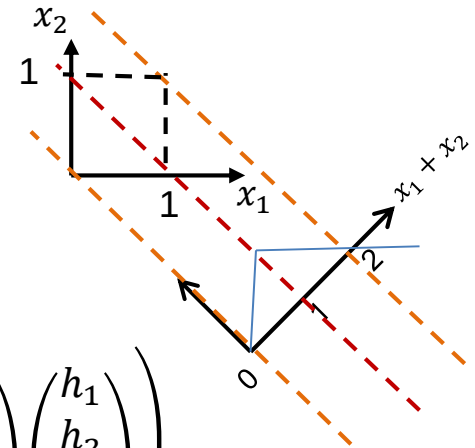
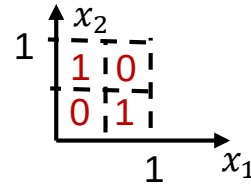
$$\mathbf{W} = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \quad \mathbf{w} = \begin{bmatrix} 1 \\ -2 \end{bmatrix}$$

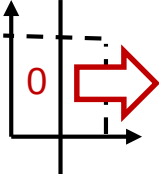
$$\mathbf{c} = \begin{bmatrix} 0 \\ -1 \end{bmatrix} \quad b = 0$$



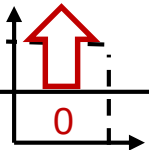
Example: Learning XOR

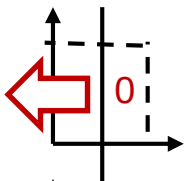
- Overfitting issue in Goodfellow's example
- Better model fitting
 - For real-circuit behavior
 - Also an example of Universal Approximation Theorem

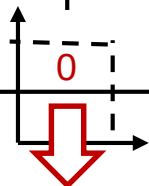


$$h_1 = \max(0, x_1 - 0.5)$$


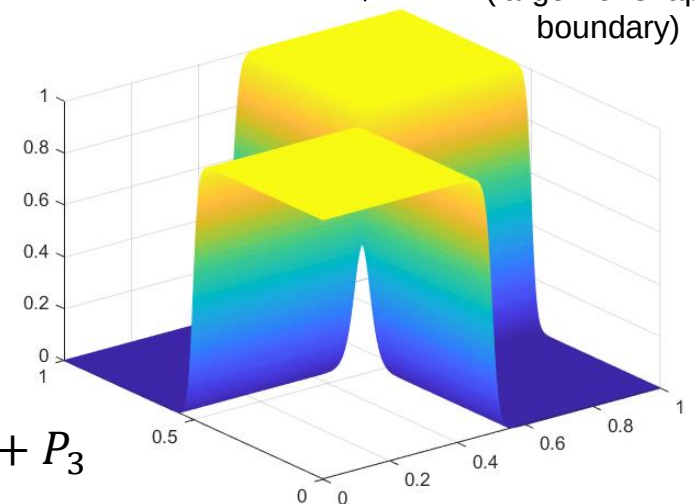
$$\begin{pmatrix} P_1 \\ P_2 \\ P_3 \\ P_4 \end{pmatrix} = \text{softmax} \left(s \begin{pmatrix} 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \\ 1 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} h_1 \\ h_2 \\ h_3 \\ h_4 \end{pmatrix} \right)$$

$$h_2 = \max(0, x_2 - 0.5)$$


$$h_3 = \max(0, 0.5 - x_1)$$


$$h_4 = \max(0, 0.5 - x_2)$$


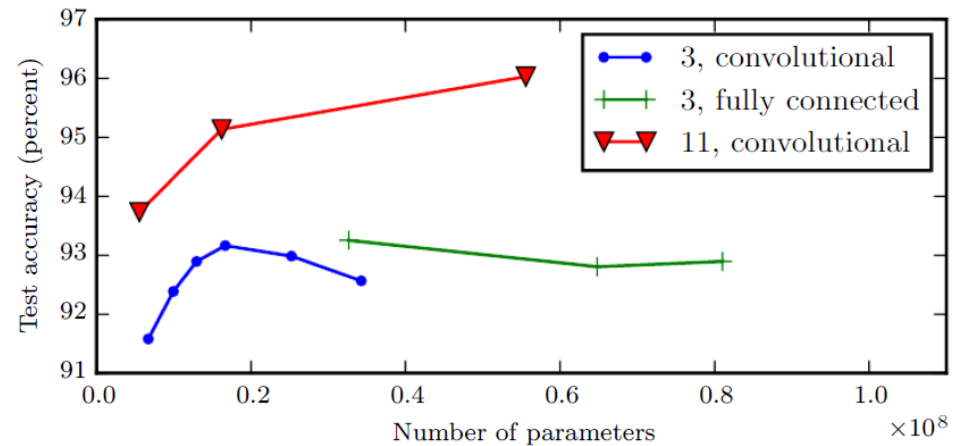
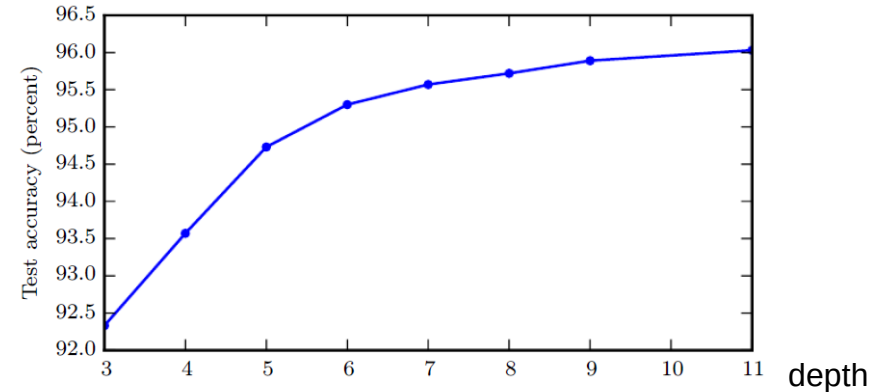
$s = 100$
(larger for sharper boundary)



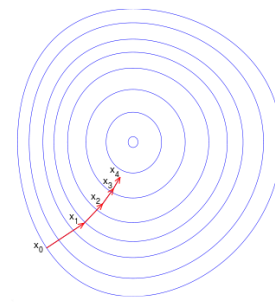
$$P(y = 0) = P_1 + P_3$$

Architectural consideration

- Depth
 - More layers, less loss
 - Training stability issue
- Weight matrix structure
 - Fully connected, convolutional, low-rank decomposition, ...
 - Try to reduce ineffective connections to reduce computation and the number of parameters
- Layer connection
 - Skip connection, ...



Gradient-based learning



- Update parameters using gradients for one single-valued loss function
 - Convergence is not guaranteed (even not expected)
 - This global loss is the sum over *all* training data D

$$L(W) = \frac{1}{|D|} \sum_i^{|D|} f_W \left(X^{(i)} \right) + \lambda r(W)$$

- In practice, we use stochastically approximated loss over mini-batch data ($N \ll |D|$) $\Rightarrow$ **Stochastic gradient descent (SGD)**

$$L(W) \approx \frac{1}{N} \sum_i^N f_W \left(X^{(i)} \right) + \lambda r(W)$$

ref: <http://caffe.berkeleyvision.org/tutorial/solver.html>



SGD with momentum

- Update parameters W by a linear combination of the negative gradient $\nabla L(W_t)$ and the previous parameter update V_t
 - Hyper-parameters: learning rate α and momentum μ

$$V_{t+1} = \mu V_t - \alpha \nabla L(W_t)$$

$$W_{t+1} = W_t + V_{t+1}$$

- Note: momentum μ effectively multiplies the update by $\frac{1}{1-\mu}$

ref: <http://caffe.berkeleyvision.org/tutorial/solver.html>



Other SGD variants

- Adam: normalize the update by estimating adaptive moments m_t and v_t

$$\begin{aligned}(m_t)_i &= \beta_1(m_{t-1})_i + (1 - \beta_1)(\nabla L(W_t))_i, \\ (v_t)_i &= \beta_2(v_{t-1})_i + (1 - \beta_2)(\nabla L(W_t))_i^2\end{aligned}$$

$$(W_{t+1})_i = (W_t)_i - \alpha \frac{\sqrt{1 - (\beta_2)_i^t}}{1 - (\beta_1)_i^t} \frac{(m_t)_i}{\sqrt{(v_t)_i} + \varepsilon}$$

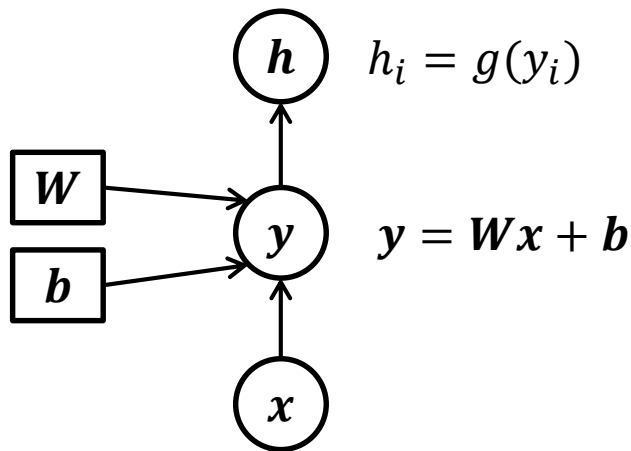
- RMSprop: normalize the update by adaptive root-mean-squared gradient

$$\begin{aligned}\text{MS}((W_t)_i) &= \delta \text{MS}((W_{t-1})_i) + (1 - \delta)(\nabla L(W_t))_i^2 \\ (W_{t+1})_i &= (W_t)_i - \alpha \frac{(\nabla L(W_t))_i}{\sqrt{\text{MS}((W_t)_i)}}\end{aligned}$$

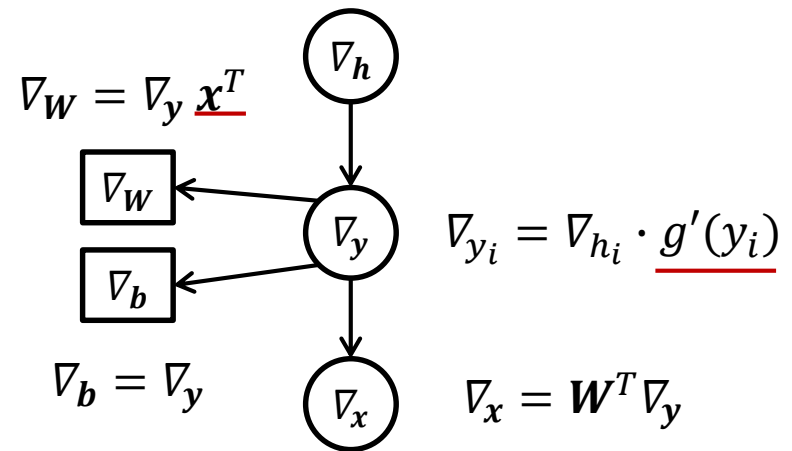
ref: <http://caffe.berkeleyvision.org/tutorial/solver.html>

Back-propagation (Backprop)

- A dynamic programming method which calculates all gradients of MLP parameters using
 - One forward-propagation pass for inference
 - for each layer: input $x \Rightarrow$ activation h
 - One backward-propagation pass for gradient calculation
 - for each layer: activation $\nabla_h L \Rightarrow$ input $\nabla_x L$ and parameter $\nabla_W L, \nabla_b L$



Forward-propagation



Back-propagation

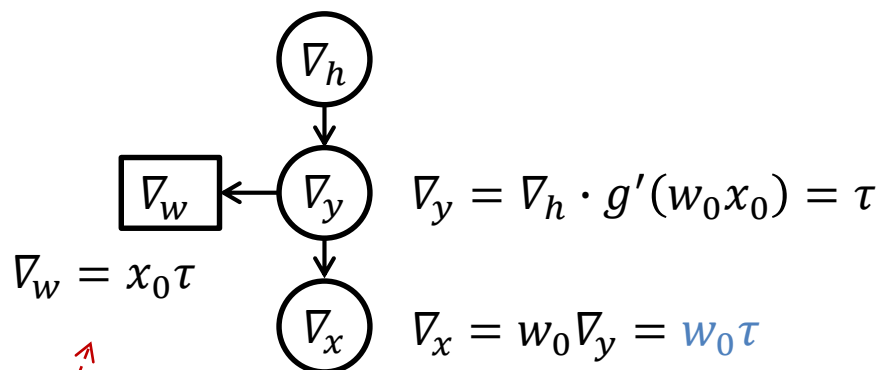
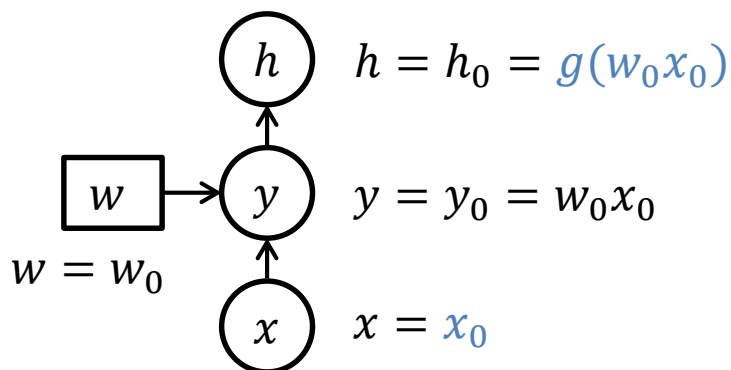


Note on gradient-based learning

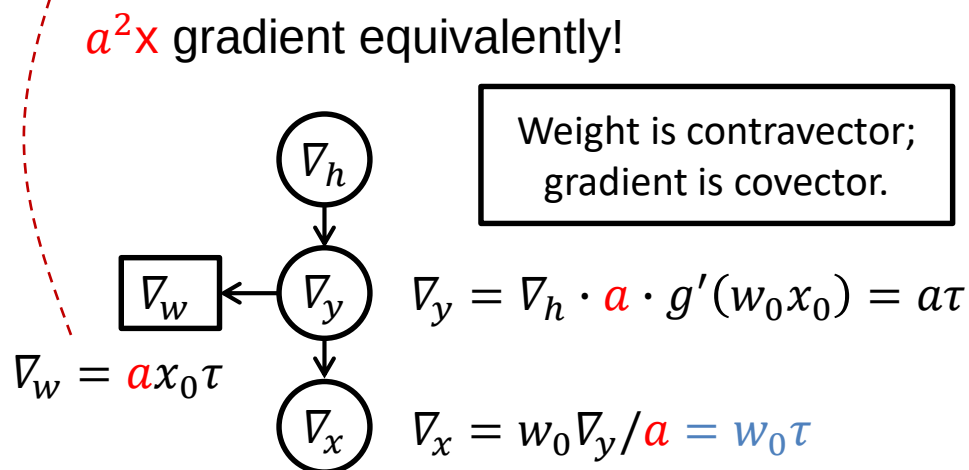
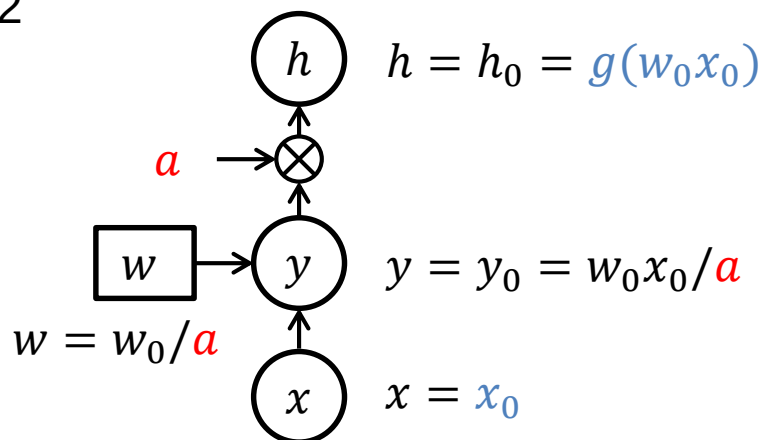
- Many training tricks are all about gradients which could explode, vanish, unbalance, ...
- Limitation
 - Convergence is not guaranteed; usually far from global minimum
 - Only for single-valued and continuous loss function
 - Backprop only works for feed-forward networks
- Two issues should be avoided
 - Vanishing gradients (e.g. ReLU activation better than sigmoid)
 - Exploding gradients (e.g. gradient normalization)
- Gradient direction more important than amplitude
 - Gradient normalization can help
 - Any good reason for one single learning rate for all parameters?

Unreliability of gradient amplitude

- Example: same inference result, but different gradient amplitude



Case 2





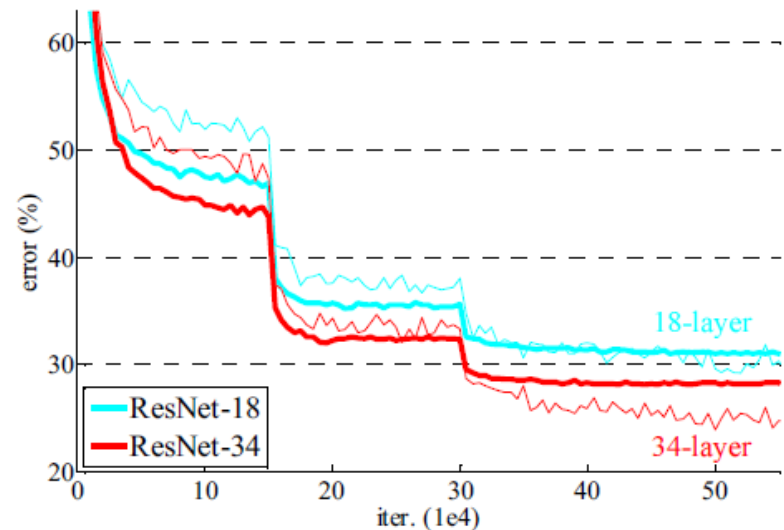
Common regularization techniques

- Weight decay
 - Dataset augmentation
 - Crop, scale, rotate, flip, ... (randomly)
 - Model averaging (Ensemble classification)
 - Early stopping with validation set

 - Noise robustness
 - Parameter sharing (e.g. convolutional)
 - Dropout
- } Based on sparse model assumption

Common optimization strategies

- Parameter initialization
 - Randomness is the key; can't be too small or too large
 - Caffe Xavier initialization $w \sim N(0, 1/n_{in})$
 - PyTorch default initialization $w \sim Unif(-1/\sqrt{n_{in}}, 1/\sqrt{n_{in}})$
- Adaptive learning rate
 - Gradually smaller
 - Step or continuous
- Batch normalization
 - Confine activation range





Batch normalization (BN)

- Solves the problem due to internal covariate shift
 - the input distribution (covariate) of each layer changes as parameters update and causes training instability
- Makes normalization as a part of the model
 - works on a mini-batch and usually before activation function

Input: Values of x over a mini-batch: $\mathcal{B} = \{x_1, \dots, x_m\}$;

Parameters to be learned: γ, β

Output: $\{y_i = \text{BN}_{\gamma, \beta}(x_i)\}$

$$\mu_{\mathcal{B}} \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad // \text{ mini-batch mean}$$

$$\sigma_{\mathcal{B}}^2 \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu_{\mathcal{B}})^2 \quad // \text{ mini-batch variance}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}} \quad // \text{ normalize}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \equiv \text{BN}_{\gamma, \beta}(x_i) \quad // \text{ scale and shift}$$

} Backprop through mini-batch mean and variance when training

} Use *ensemble* mean and variance when testing

} γ and β are learnable parameters

BN transform



Advantages of BN

- Accelerating and stabilizing the training procedure

- ∇_x are normalized as well

$$\frac{\partial \ell}{\partial x_i} = \frac{\partial \ell}{\partial \hat{x}_i} \cdot \frac{1}{\sqrt{\sigma_B^2 + \epsilon}} + \frac{\partial \ell}{\partial \sigma_B^2} \cdot \frac{2(x_i - \mu_B)}{m} + \frac{\partial \ell}{\partial \mu_B} \cdot \frac{1}{m}$$

- Larger weights lead to smaller gradients

- Higher learning rate
- Less careful parameter initialization

$$\frac{\partial \text{BN}((aW)u)}{\partial u} = \frac{\partial \text{BN}(Wu)}{\partial u}$$

$$\frac{\partial \text{BN}((aW)u)}{\partial (aW)} = \frac{1}{a} \cdot \frac{\partial \text{BN}(Wu)}{\partial W}$$

- Regularization through the perturbation of mini-batch mean and variance

- Experimental result

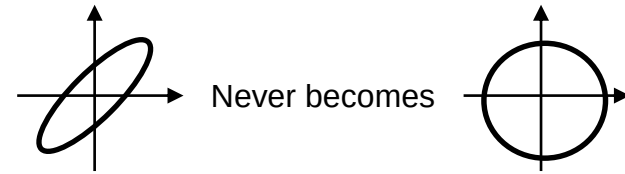
- 14x faster training with better accuracy
- Eliminates the need of Dropout

- Implementation detail

- Can merge BN transform into weights and biases when testing

Notes on BN

- Stabilizing covariate?



- Shielding parameters from other layers

$$\frac{\partial^2 l}{\partial w_i \partial w_j} \cong 0 \text{ if } w_i \text{ and } w_j \text{ are not in the same layer.}$$

- Enabling larger learning rates

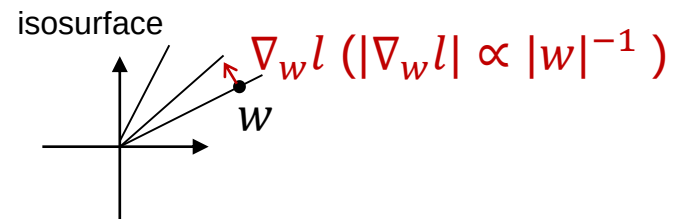
$$l(w - \epsilon g) \cong l(w) - \underbrace{\epsilon \nabla_w l^t \cdot g}_{\text{this term can then be increased for faster optimization}} + \underbrace{\frac{1}{2} \epsilon^2 g^t H g}_{\text{this term may lead to loss increase but now is reduced}} \text{ where } H_{ij} = \frac{\partial^2 l}{\partial w_i \partial w_j}$$

this term can then be increased for faster optimization

this term may lead to loss increase but now is reduced

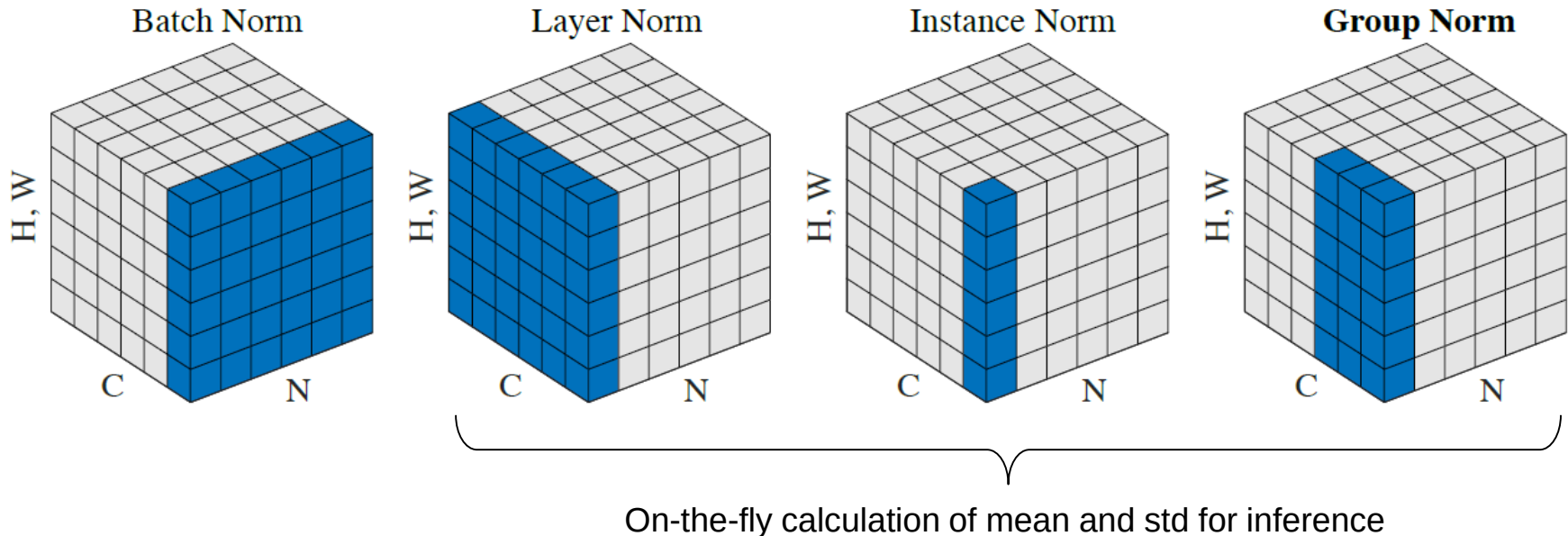
- Avoiding meaningless update

$$\nabla_w l \perp w \because BN(aw) = BN(w)$$



- BN(wx) vs. wBN(x)?

Variants of normalization



All satisfy the weight re-scaling property: $XN(a(Wu)) = XN(Wu)$.

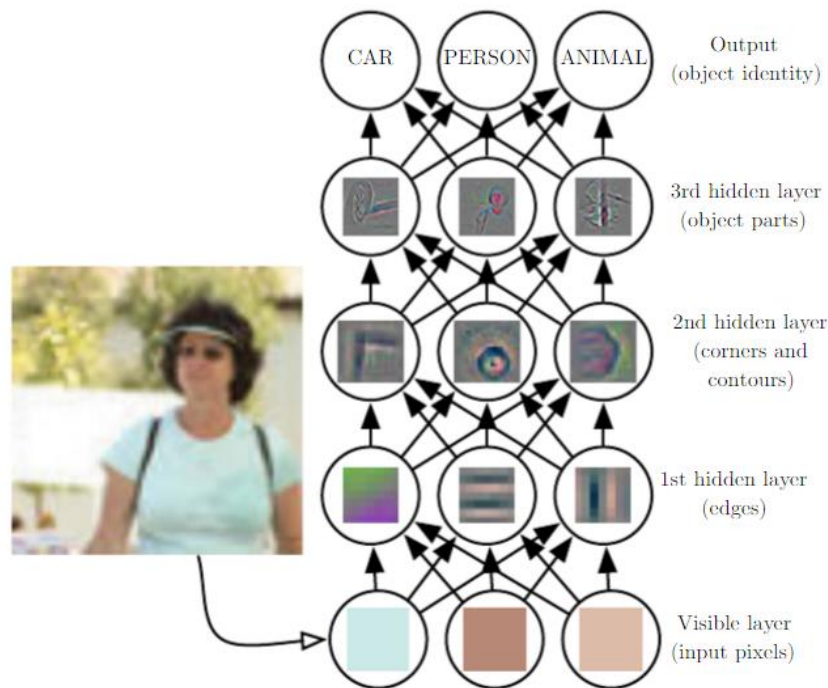


Outline

- Machine learning basics
- Deep feed-forward network training
- **CNN evolution for image recognition**
 - CNN architecture
 - CNN evolution

CNN architecture

- Use strong prior to constrain networks for vision applications with
 - much fewer parameters using convolution
 - much fewer features (activations) using striding and pooling

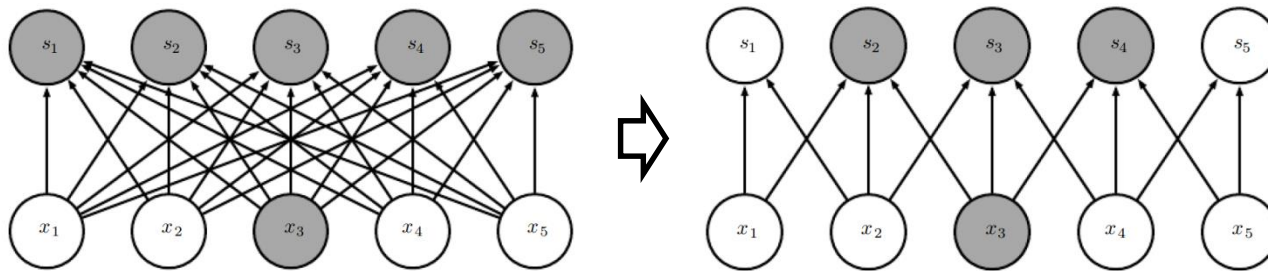


Higher-layer activations contain higher-level features with coarser resolution
⇒ Use striding and pooling

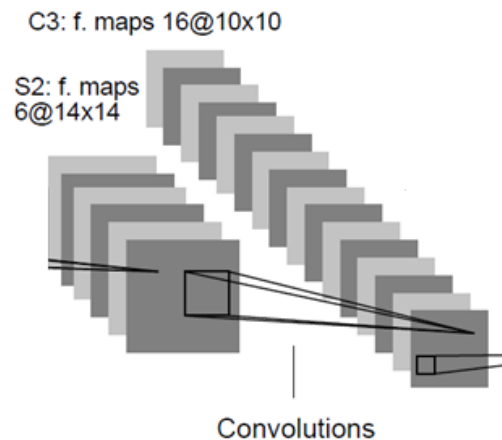
Visual features should only correlate with local neighbors ⇒ Use convolution

Convolution (CONV) layer

- Turn dense matrix connection, fully connected (FC) layer, into sparse connectivity

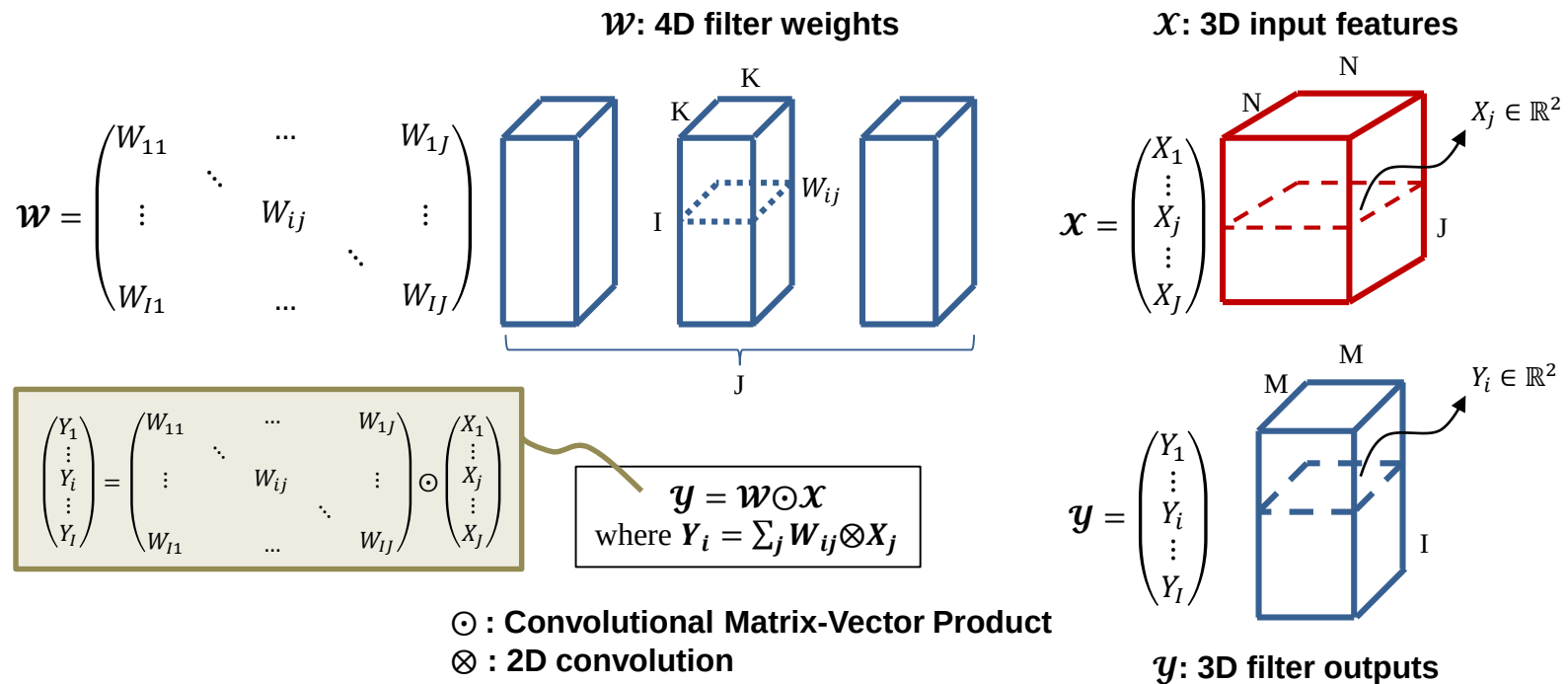


- It is tensor filtering, not the conventional convolution



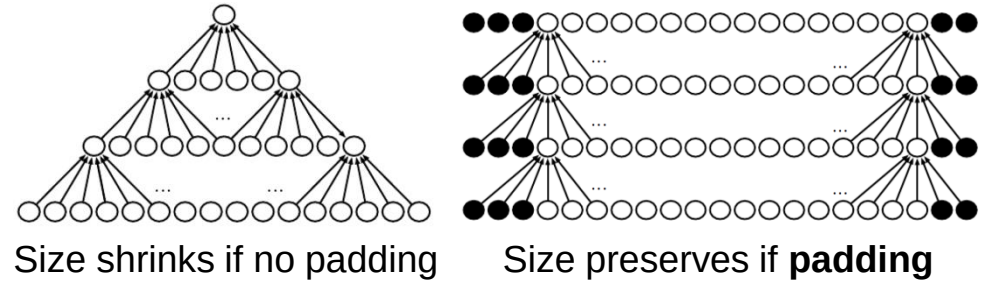
Convolutional matrix-vector product

- Another way to express tensor filtering

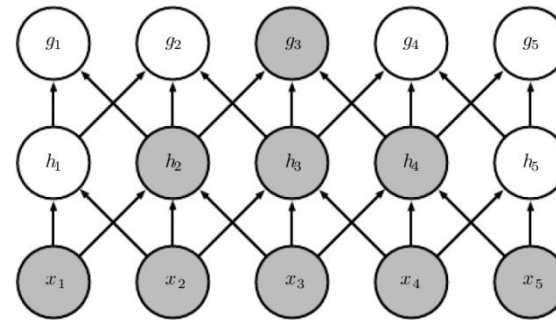


Important settings of CONV layer

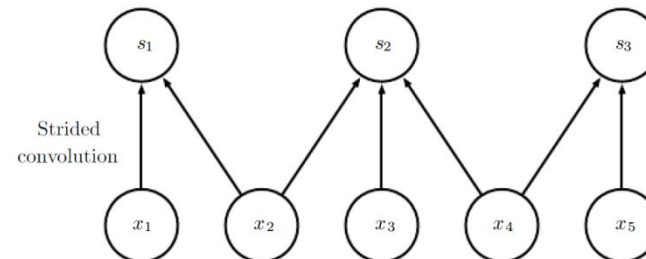
- Filter size
- Padding
 - Zero padding is the norm



- Receptive field
 - Spatial range of input pixels for one output feature

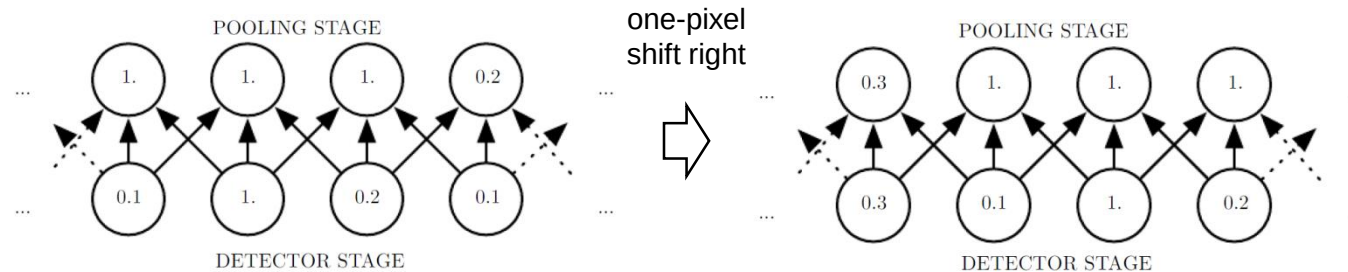


- Stride
 - Example: stride size is two

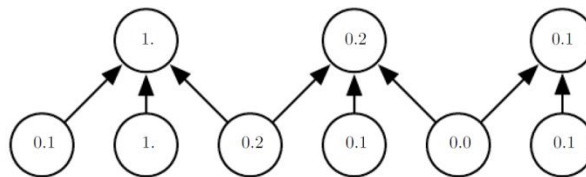


Pooling layer

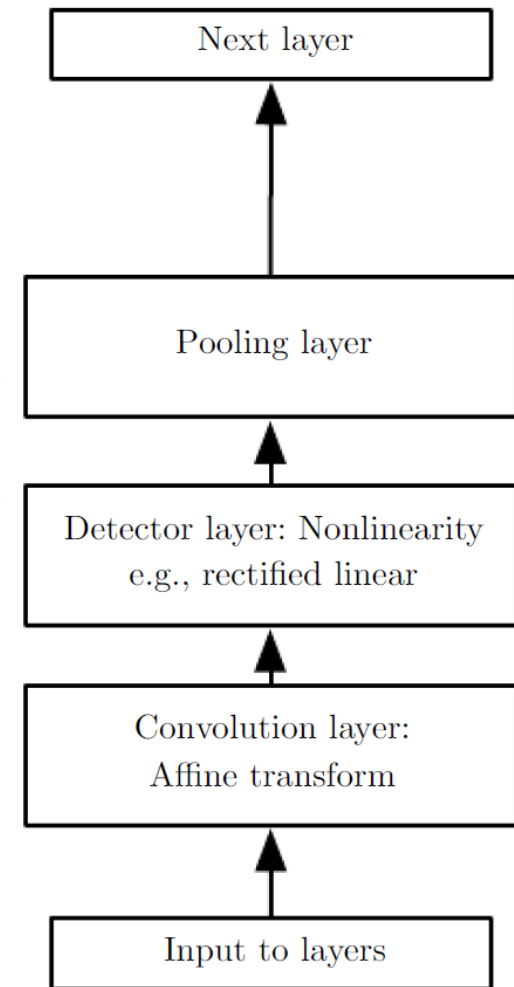
- Usually applied with striding to reduce spatial size of features
 - Max pooling: provide spatial invariance



- Example: stride size is two



- Average pooling: aggregate features



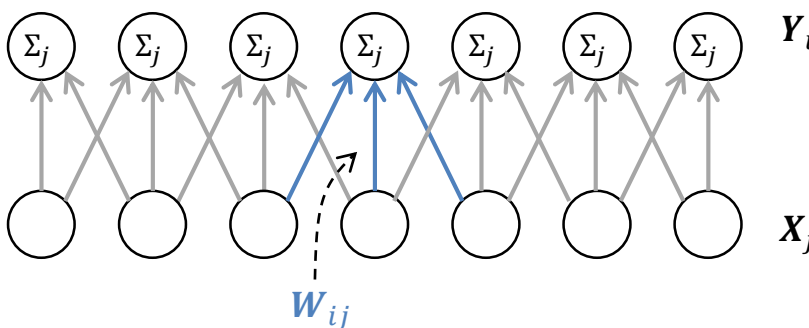


Backprop for CNN

- Similar to the case of MLP
 - Backprop convolution uses zero padding

Forward-propagation

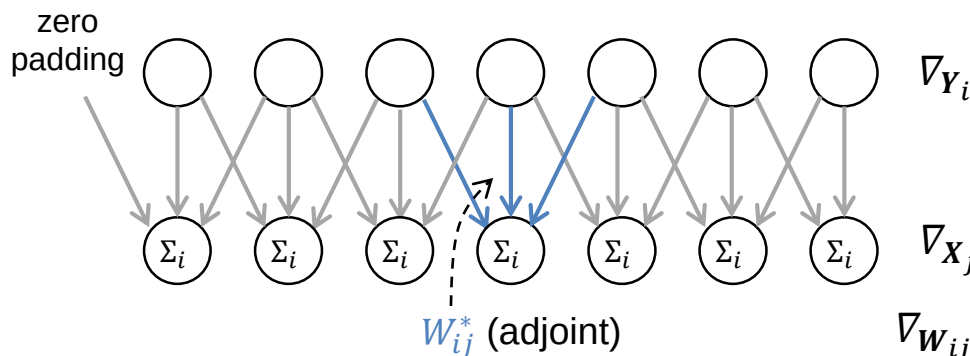
$$\text{CMVP: } \mathbf{y} = \mathbf{w} \odot \mathbf{x}$$



$$Y_i = \sum_j w_{ij} \otimes X_j$$

Back-propagation

$$\begin{aligned} \text{CMVP:} \\ \nabla_{\mathbf{x}} &= \mathbf{w}^{*T} \odot \nabla_{\mathbf{y}} \\ \nabla_{\mathbf{w}} &= (\mathbf{x} \star \nabla_{\mathbf{y}}^T)^T \end{aligned}$$



$$\begin{aligned} \nabla_{X_j} &= \sum_i w_{ij}^* \otimes \nabla_{Y_i} \\ \nabla_{w_{ij}} &= X_j \star \nabla_{Y_i} \\ &\quad \text{(cross-correlation)} \end{aligned}$$



Outline

- Machine learning basics
- Deep feed-forward network training
- CNN evolution for image recognition
 - CNN architecture
 - **CNN evolution**

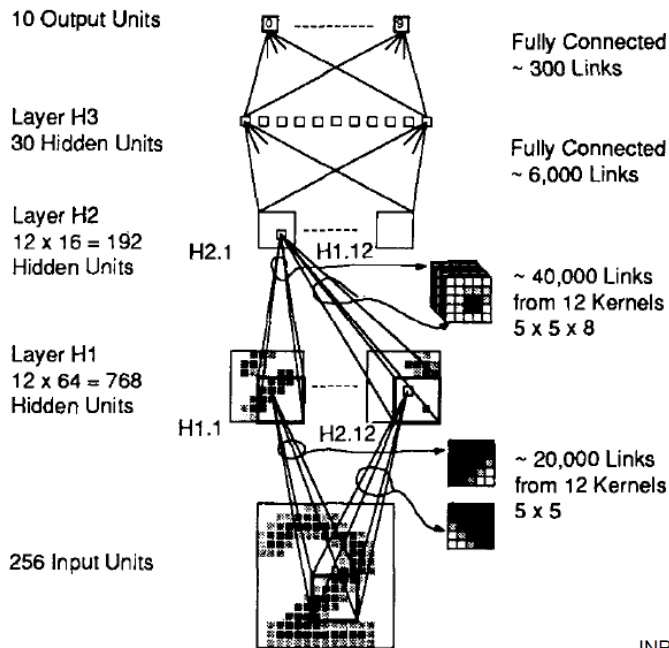
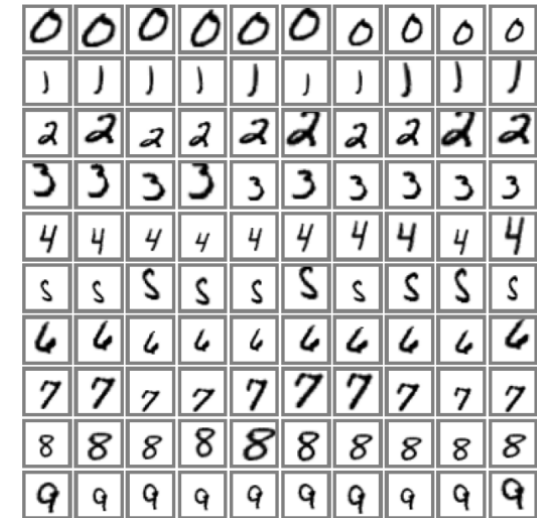
Long-term contribution:

- CONV layer
- Pooling

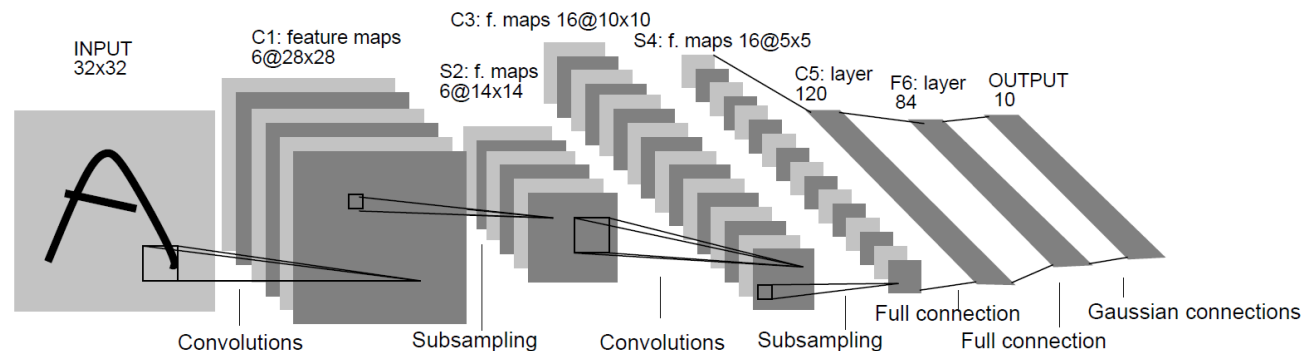
LeNet

- First CNN (digit recognition)

MNIST dataset



LeNet-5 (1998)
5 layers: 2 CONV, 3 FC
Test error rate: <1%



LeNet (1989)

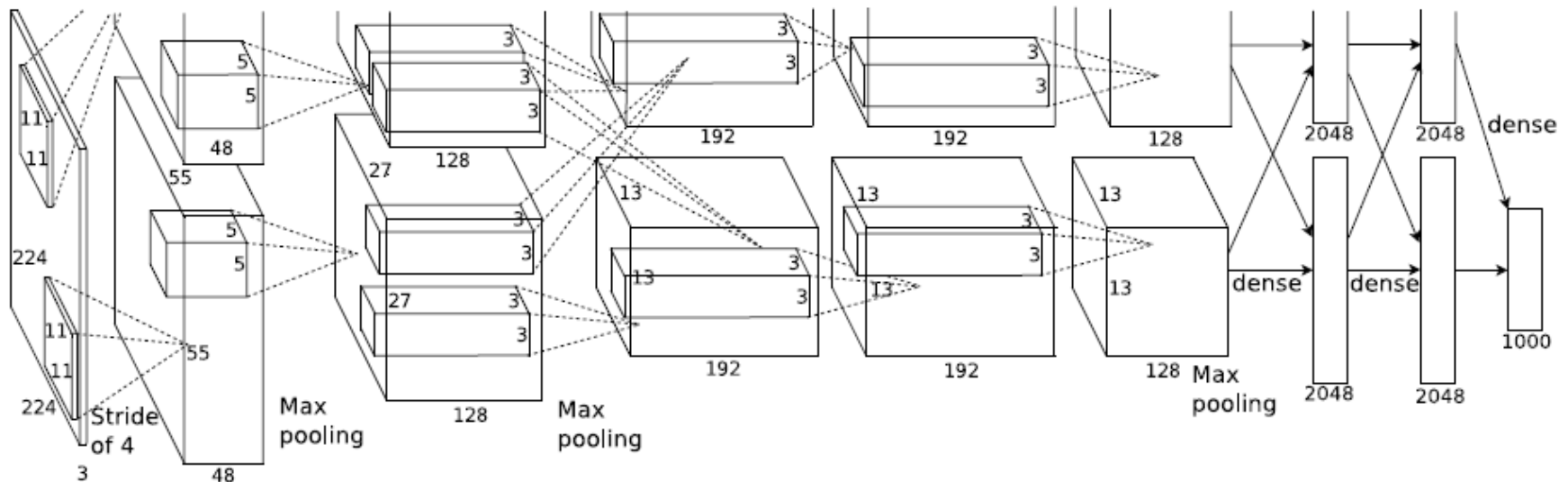
4 layers: 2 CONV, 2 FC
Test error rate: 5%

Long-term contribution :

- ReLU
- Data augmentation
- ImageNet end-to-end learning
- GPU acceleration

AlexNet (2012)

- First “deep” CNN
 - 8 layers: 5 CONV, 3 FC
 - Other techniques: Dropout, local contrast normalization
 - Impressive accuracy improvement for ImageNet (1000 classes)
 - Top-5 test error: 16.4% (5 CNNs); previous best work: 26.2%
 - Top-5 validation error: **18.2%** (1 CNN); 16.4% (5 CNNs)
 - Top-1 validation error: **40.7%** (1 CNN); 38.1% (5 CNNs)



Long-term contribution :

- Small filter (3x3, 1x1)



VGGNet (2015)

- Small filter size to go deeper
 - 11-19 layers: 3 FC
 - Use only 3x3 and 1x1 (bottleneck) filters
 - Impressive accuracy improvement for ImageNet
 - Top-5 val. error: **8.0%** (VGG-19)
 - Top-1 val. error: **25.5%** (VGG-19)
 - Top-5 val. error: 6.8% (VGG-16/19*)
 - Top-1 val. error: 23.7% (VGG-16/19*)
 - *: multi-crop, dense eval.

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Spatial size shrinks,
feature channel grows

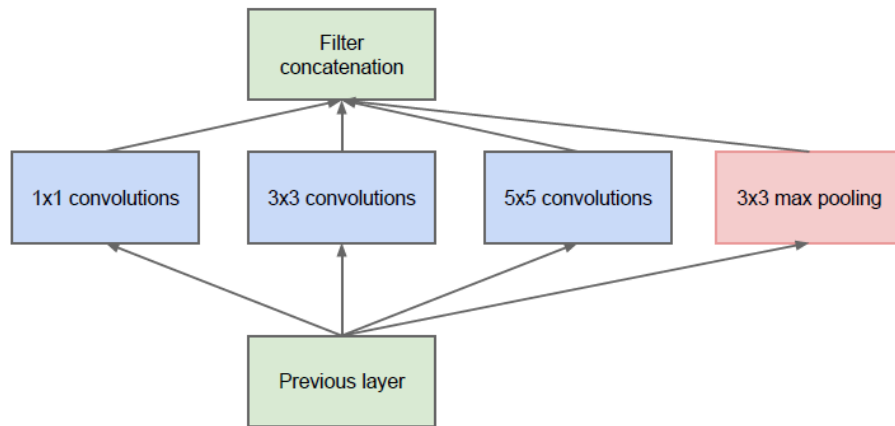


- 1 FC only

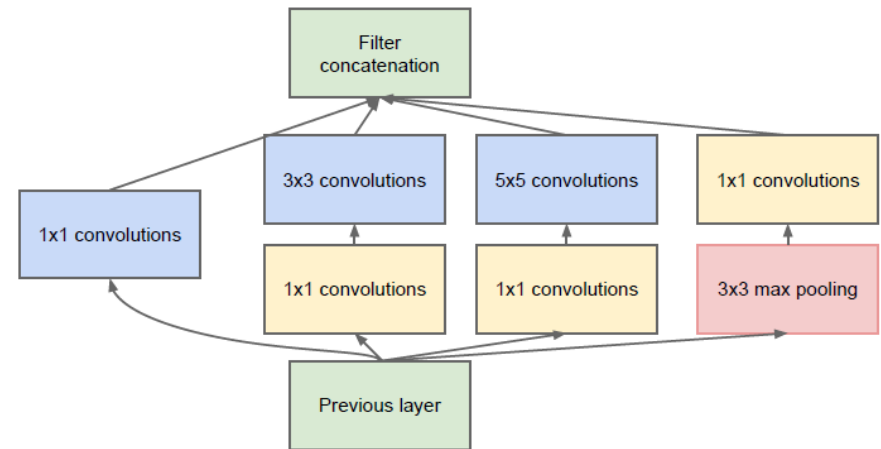


GoogLeNet (2015)

- Inception module to reduce complexity
 - 22 layers: 1 FC; average pooling layer
 - Accuracy for ImageNet
 - Top-5 test error: **10.1%** (GoogLeNet-22)
 - Top-5 test error: 6.7% (144 crops, 7 CNNs)



(a) Inception module, naïve version

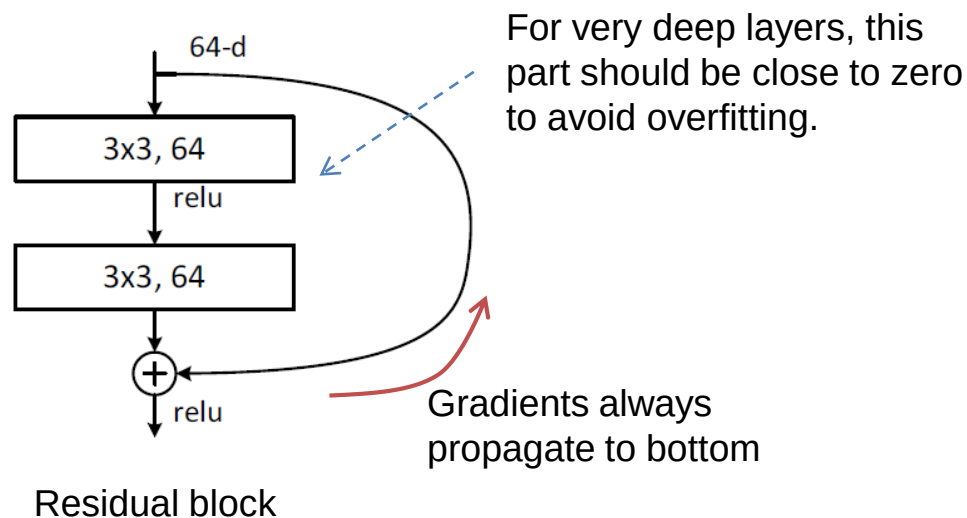


(b) Inception module with dimensionality reduction



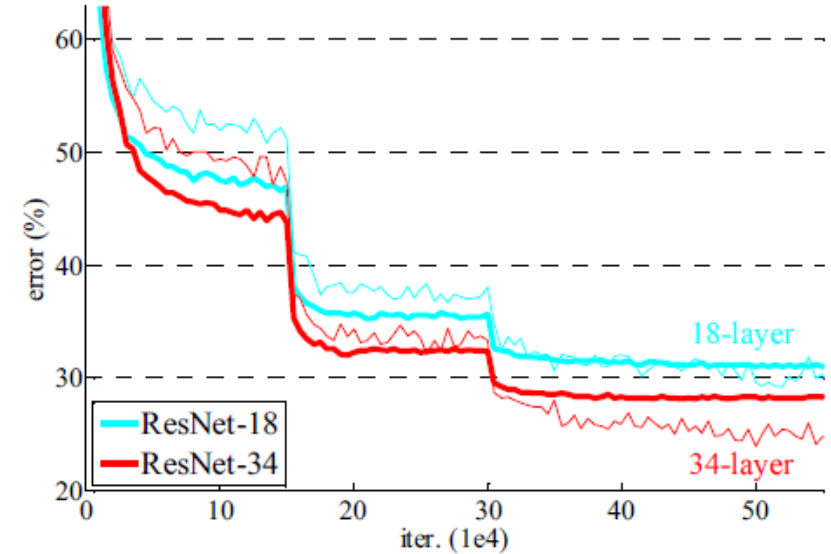
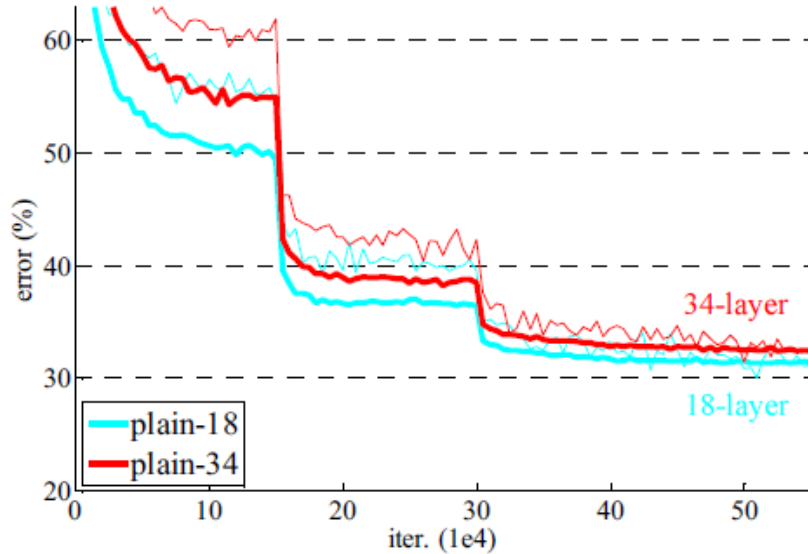
ResNet

- Residual block to stabilize training and enable deeper layers (up to 1000 layers!)
 - 1 FC layer
 - Accuracy for ImageNet
 - Top-5 val. error: 5.7%/ 5.3%/ 4.5% (ResNet-34/50/152, 144 crops)
 - Top-1 val. error: 21.8%/20.7%/19.4% (ResNet-34/50/152, 144 crops)



Residual network

- Make deeper networks always better than shallower ones



Thin: training error
Bold: validation error



Robustness (limit) of deep networks

- The features learned by deep networks are different from what people learn and could be fragile in real world
- Well-known attacks for recognition networks
 - Additive perturbations
 - Adversarial (Image-dependent), e.g. $\nabla_{\mathbf{x}} L$
 - Universal (image- or even model-independent)
 - Random noise
 - Geometric transformations

Ref: A. Fawzi et al, “The Robustness of Deep networks,” IEEE SPM, 2017.

[optional]



Example

Original
(CaffeNet)



Pomeranian

Adversarial
perturbation



Marmoset

Random
noise



Marmoset

Semi-random
noise



Marmoset

Universal
perturbation



Mosquito Net

Affine
transformation



Persian Cat



Universal
perturbation
for GoogLeNet



(a) Wool



(b) Indian Elephant



(c) Indian Elephant



(d) African Gray



(e) Tabby



(f) African Gray



(g) Common Newt



(h) Carousel



(i) Gray Fox



(j) Macaw

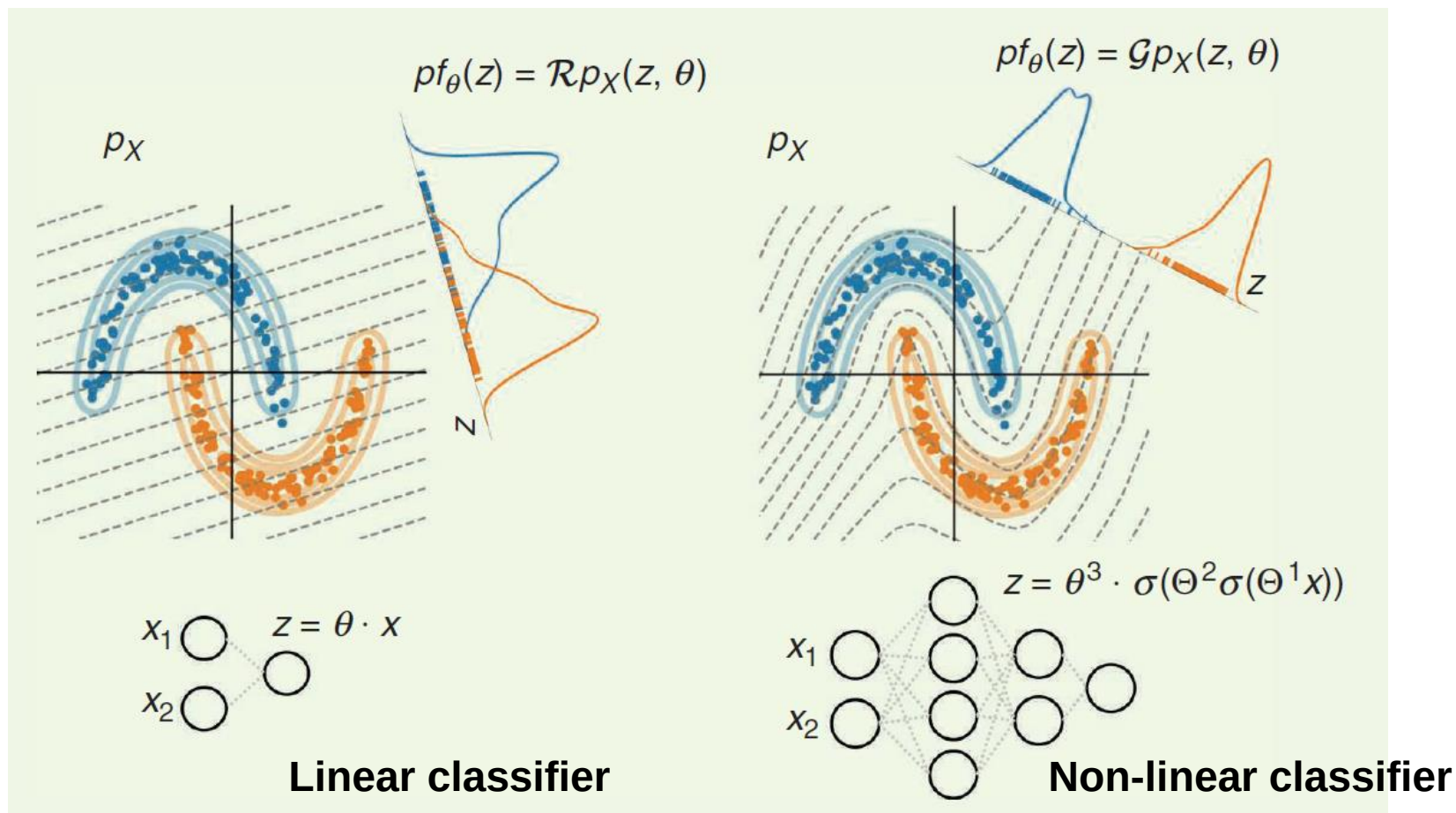


(k) Three-Toed Sloth



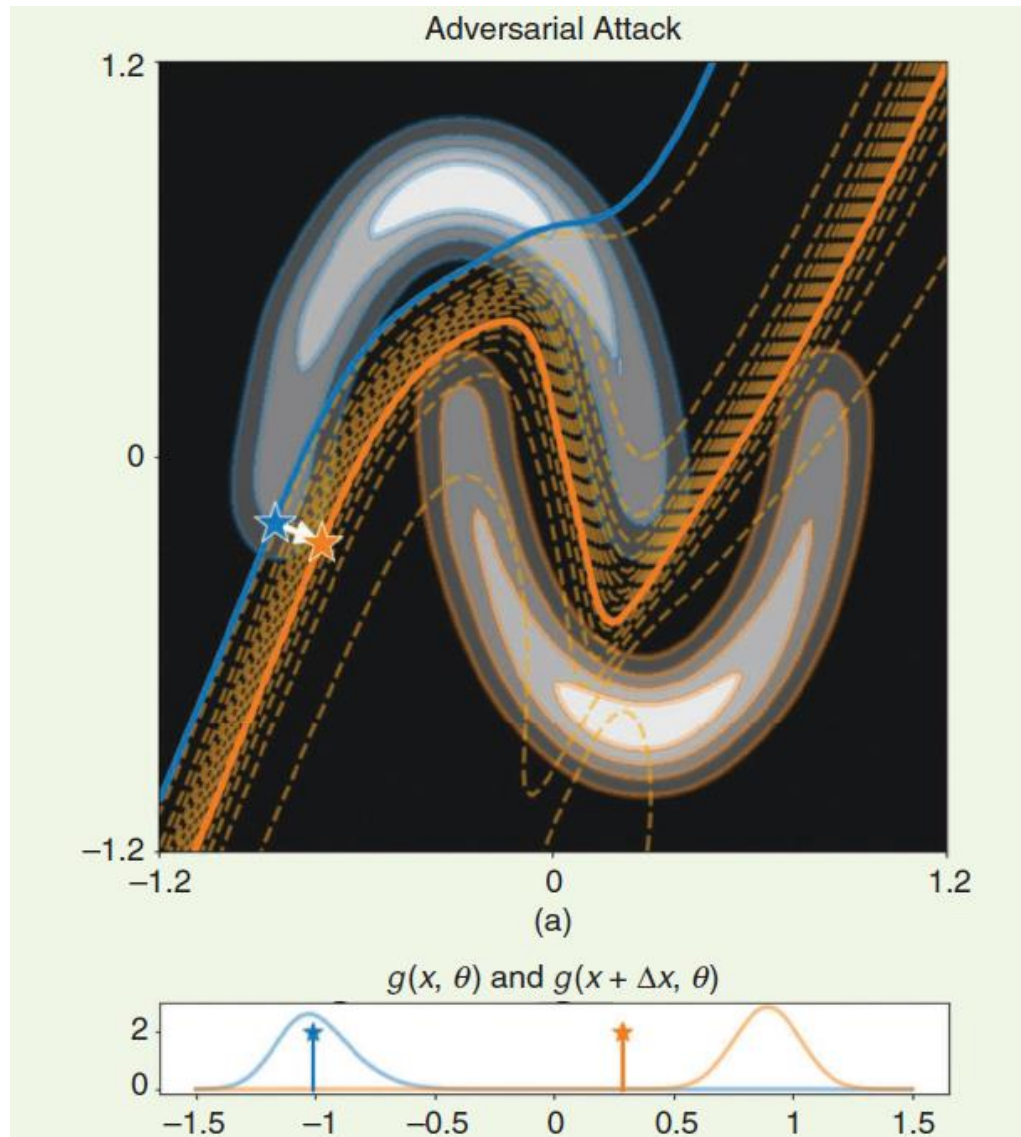
(l) Macaw

A view from geometry: isosurface fitting



Ref: S. Kolouri et al, "Neural Networks, Hypersurfaces, and the Generalized Radon Transform," IEEE SPM, 2020.

Explanation of adversarial attack





Appendix



Logical reasoning

- Deduction

- Premises $\xrightarrow{\text{rules}}$ Conclusions
- Logical operations

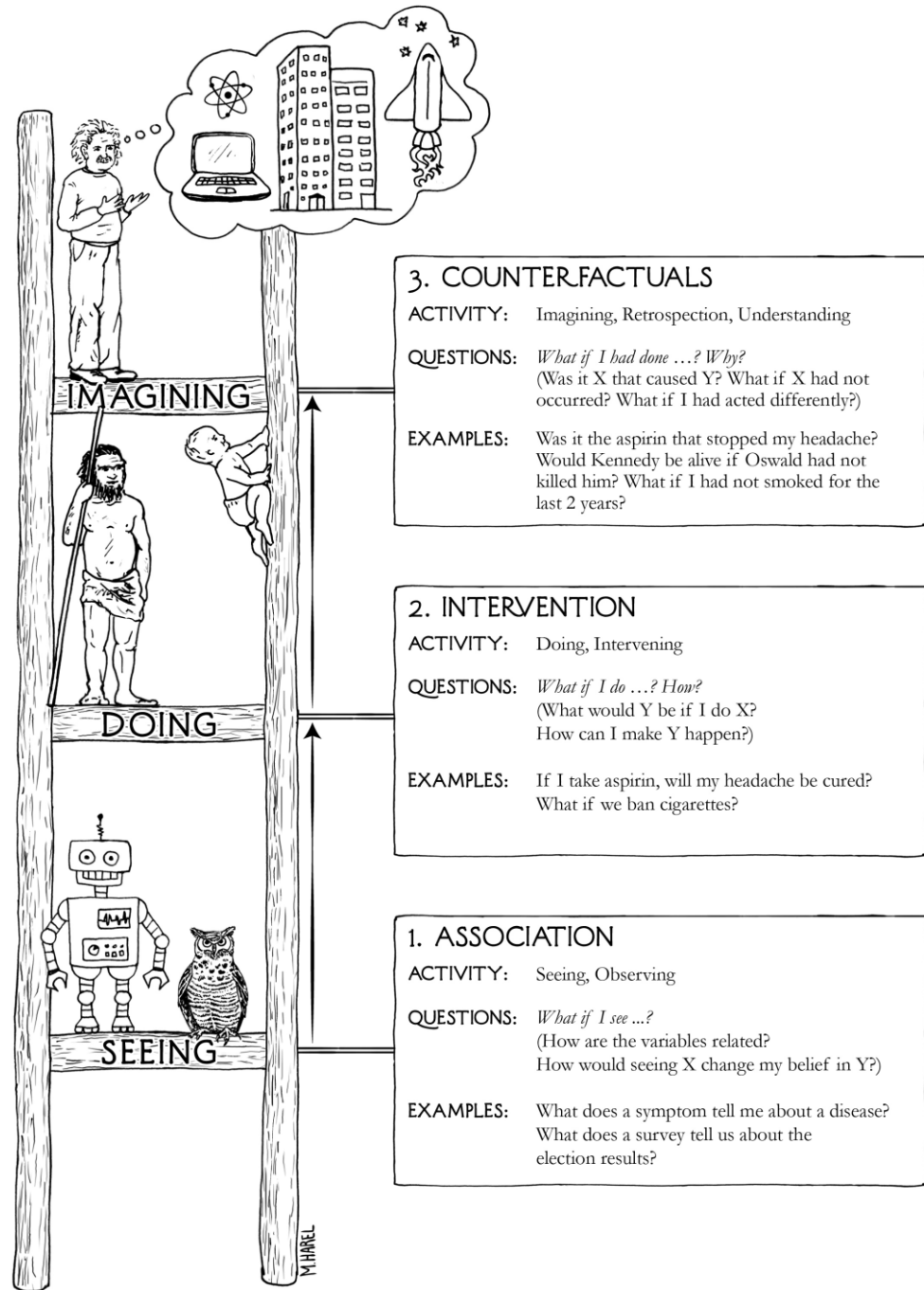
- Induction

- Observations $\longrightarrow$ Hypotheses $\xrightarrow[\text{(science)}]{\text{Experiments (randomized, controlled)}} \text{Rules}$
- Scientist, researcher

- Abduction

- Conclusions $\xrightarrow{\text{best guess}}$ Premises
- Detective, machine learning

Ladder of causation



[Judea Pearl, *The Book of Why*.]